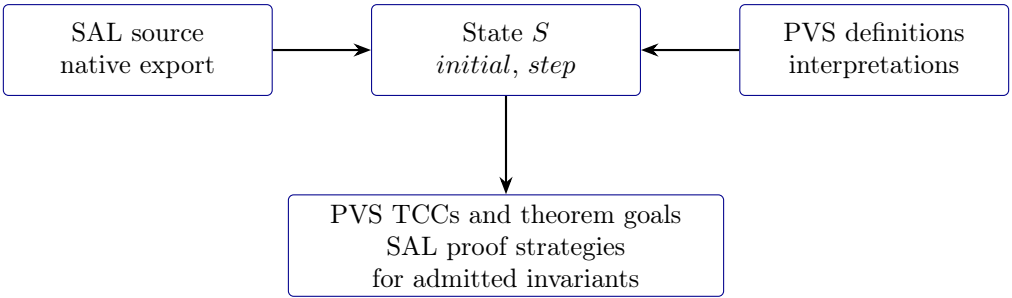
SAL and PVS: a shallow bridge

A tutorial from source-level TCCs to native PVS model checking

Experimental integration | src/sal

October 2026

Two languages, one transition-system view



Native records, functions and predicates carry the semantics. There is no object-language interpreter inside the PVS logic.

The route through this tutorial

1. Import a SAL machine; inspect a source-associated typing failure.
2. Add parameters, imports, composition and progress contracts.
3. Interpret an existing PVS library; extract an admissible SAL machine.
4. Prove safety, inspect a counterexample, and exercise the refusal checks.

What is implemented

Both frontends share the transition semantics. Proof closure currently admits finite safety invariants through an experimental trusted SAL rule.

Start a local demonstration

Set `PVS_BIN`, `SAL_HOME` and `SAL_XMLS_SOURCE` to existing installations.

```
# From the repository root:
src/sal/doc/run-demo.sh
```

The launcher loads `doc/demo.lisp` into a raw PVS session. The helpers below copy or generate files in `doc/.demo/`; they do not prove TCCs for you.

```
;; PVS Lisp: import an existing SAL example.
(defparameter v
  (sal-doc-open "bounded_counter.sal" "counter" "counter"))
```

Prerequisites and interactive Emacs export: doc/README.org

The semantic contract includes terminal states

```
SAL_Transition_System[S: TYPE,
  initial: [S -> bool], step: [S, S -> bool]]: THEORY
```

A state is reachable when a finite prefix starts in `initial`, follows `step`, and ends there.

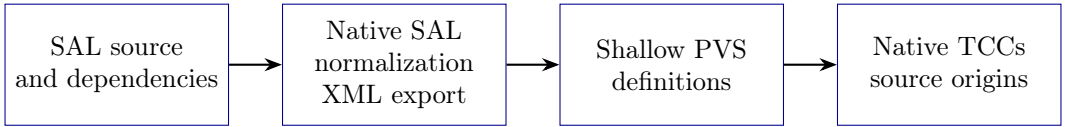
```
invariant(p: [S -> bool]): bool =
  FORALL (s: S): reachable(s) IMPLIES p(s)

deadlock_free: bool =
  FORALL (s: S): reachable(s) IMPLIES
    EXISTS (t: S): step(s, t)
```

Safety sees a reachable terminal state. Progress is a separate obligation.

theories/SAL_Transition_System.pvs; declarations excerpted

SAL enters through its native frontend



- SAL resolves modules and dependency contexts.
- PVS receives ordinary state types, initial predicates and step relations.
- Separate assignment and command-choice checks retain obligations that would otherwise be masked by overlapping branches.

Counter: state and initialization

The `library_counter` example. Here `defined` is `TRUE`; PVS formulas omit it, neutral conjuncts and redundant parentheses.

SAL module excerpt

```
machine: MODULE =
BEGIN
  OUTPUT count: [0..3]
  OUTPUT started: BOOLEAN
  INITIALIZATION
    count = 0;
    started = FALSE
  % Transitions follow.
END;
```

PVS module theory excerpt

```
State: TYPE =
  [# sal_count: subrange(0, 3),
     sal_started: bool #]

initial(s: State): bool =
  s`sal_count = 0 AND
  s`sal_started = FALSE
```

Each SAL variable becomes a field of an ordinary PVS record.

Counter: transitions and safety

The same example. `defined` is `TRUE` here; it and neutral conjuncts are omitted. Complete, unchanged translations are in `doc/counterparts/counter/`.

SAL guarded commands

```
TRANSITION
 [ advance: count < 3 -->
     count' = count + 1;
     started' = TRUE
 [] hold: ELSE -->
     count' = count;
     started' = started ]

safe: THEOREM machine |-
  G(count = 0 OR started);
```

PVS relation and invariant

```
step(s, next: State): bool =
 (s`sal_count < 3 AND
  next`sal_count = s`sal_count + 1 AND
  next`sal_started = TRUE) OR
 (NOT (s`sal_count < 3) AND
  next`sal_count = s`sal_count AND
  next`sal_started = s`sal_started)

IMPORTING SAL_Transition_System[
  State, initial, step] AS system
sal_safe: THEOREM system.invariant(
  LAMBDA (s: State):
    s`sal_count = 0 OR s`sal_started)
```

First inspection: the generated theory and TCCs

```
;; v is the sal-transition-view returned by sal-doc-open.
(sal-transition-view-pvs-file v)
(sal-show-obligations v)

;; Use a declaration/TCC name from that report.
(sal-show-source v "check_step_5_TCC1")
```

The generated `.pvs` and `.tccs` files are ordinary PVS artifacts. The report connects native formulas to SAL filenames, spans and expressions.
The exact TCC names depend on the selected example. Next, open the overflow example, whose report contains `check_step_2_TCC1`.

A valid branch does not hide an invalid branch

```
Small: TYPE = [0..2];
% Inside the machine:
OUTPUT value: Small
TRANSITION
  [ safe: TRUE --> value' = 0
  [] overflow: TRUE -->
       value' = value + 1 ]
```

Both guards can hold. The second branch must still satisfy:

$$0 \leq value + 1 \ \wedge \ value + 1 \leq 2$$

At $value = 2$, the obligation is false. The successful reset branch does not justify the overflow branch.

See the failure in its SAL source

```
(defparameter v
  (sal-doc-open "integration/overlapping_subtype.sal"
                "machine" "overflow"))
(sal-show-obligations v)
(sal-show-source v "check_step_2_TCC1")
```

Recorded report, with the repository prefix omitted:

```
overlapping_subtype.sal:15 check_step_2_TCC1 [SUBTYPE; NIL]
  SAL: value' = value + 1
  PVS: FORALL (s, next: State):
       0 <= (s`sal_value + 1) AND (s`sal_value + 1) <= 2
```

Typechecking creates the obligation; it does not establish the property.

Parameterized contexts stay parameterized

```
parameter_machine{T: TYPE, Limit: NATURAL}: CONTEXT =
BEGIN
  Count: TYPE = [0..Limit];
  identity(x: T): T = x;
  cell[seed: T, initial_count: Count]: MODULE =
    BEGIN
      OUTPUT payload: T
      OUTPUT count: Count
      INITIALIZATION payload = identity(seed);
                     count = initial_count
      % Transition declarations omitted from this excerpt.
    END;
END
```

Context and module parameters become shallow PVS theory parameters. Generic typechecking does not silently choose a model-checker instance.

Parameterized context: the generated PVS library

These are source and generated context declarations, with line wrapping only.

SAL context excerpt

```
parameter_machine{
  T: TYPE, Limit: NATURAL
}: CONTEXT =
BEGIN
  Count: TYPE = [0..Limit];
  identity(x: T): T = x;
  % Parameterized cell follows.
END
```

The module theory adds `seed` and `initial_count` formals, retaining the latter's type from this instantiated context. See `counterparts/parameters/`.

PVS context theory excerpt

```
SALCTX_sal_parameter_x5F_machine[
  sal_T: TYPE, sal_Limit: nat]: THEORY
BEGIN
  sal_Count: TYPE =
    subrange(0, sal_Limit)
  sal_identity: [sal_T -> sal_T] =
    (LAMBDA (bridge_local_1_sal_x:
                sal_T):
       bridge_local_1_sal_x)
END SALCTX_sal_parameter_x5F_machine
```

Choose an instance explicitly

```
(defparameter generic
  (sal-doc-open "parameters/parameter_machine.sal"
                "cell" "generic"))

(defparameter concrete
  (sal-doc-open "parameters/parameter_machine.sal"
                "cell" "concrete"
                :context-actuals '("BOOLEAN" "2")
                :module-actuals '("FALSE" "0")))
```

Equivalent native selector: `parameter_machine{BOOLEAN, 2}!cell[FALSE, 0]`.
SAL instantiates and normalizes that selection. Actual-argument subtype obligations remain visible even for parameters unused by the transition.

Imported contexts retain their identities

```
left: CONTEXT = parameter_left{BOOLEAN, 2};
right: CONTEXT = parameter_right{BOOLEAN, 2};
% In the transition:
count' = right!advance(left!advance(count));
```

```
(defparameter imports
  (sal-doc-open "parameters/parameter_diamond.sal"
                "system" "diamond"))
(sal-show-obligations imports)
```

- The transitive closure includes shared and selector-only dependencies.
- Distinct actuals remain distinct instances of one library.
- An imported TCC points to the declaring SAL file; snapshots cover the closure.
- `sal-open-context` imports a library-only context without inventing a machine.

N-ary composition: a parameterized SAL family

The worked model `doc/nary/nary.sal` has a positive number of identical cells.

```
nary{N: {n: NATURAL | n > 0}}: CONTEXT =
BEGIN
  Index: TYPE = [1..N];

  cell: MODULE =
    BEGIN
      OUTPUT value: [0..2]
      INITIALIZATION value = 0
      TRANSITION
        [ increment: value < 2 --> value' = value + 1
        [] wrap: ELSE --> value' = 0 ]
    END;
```

Each cell cycles $0 \to 1 \to 2 \to 0$. Its initialization and deterministic step will let us establish a relation between different cells.

N-ary composition: wiring and the safety claim

Inside the same parameterized context:

```
sync: MODULE =
  WITH OUTPUT values: ARRAY Index OF [0..2]
    (|| (i: Index): RENAME value TO values[i] IN cell);

async: MODULE =
  WITH OUTPUT values: ARRAY Index OF [0..2]
    ([] (i: Index): RENAME value TO values[i] IN cell);

aligned: THEOREM sync |-
  G(FORALL(i, j: Index): values[i] = values[j]);
```

For `N = 3`, synchronous execution follows $(0,0,0) \to (1,1,1) \to (2,2,2) \to (0,0,0)$. `aligned` says all components agree. This follows from equal initialization, identical deterministic behavior and synchronous composition together.

N-ary composition: the parameterized PVS counterpart

Generated context and module excerpts. Local subtype/index binders are renamed; `defined` (a universally true predicate) and neutral conjunctions are omitted.

```
SALCTX_sal_nary[sal_N: {n: nat | n > 0}]: THEORY
  % In this context's body:
  sal_Index: TYPE = subrange(1, sal_N)

SAL_sal_nary_sal_sync[sal_N: {n: nat | n > 0}]: THEORY
  % In this module theory's body:
  IMPORTING SALCTX_sal_nary[sal_N]
  State: TYPE = [# sal_values: [sal_Index -> subrange(0, 2)] #]
  initial(s: State): bool =
    FORALL (i: sal_Index): (s`sal_values)(i) = 0
```

`N` remains a theory parameter; the array becomes a PVS function. Complete generated theories: `doc/nary/generated/generic/`.

N-ary composition: one quantified transition relation

Same generic module theory; local binders renamed and neutral predicates removed.

```
step(s, next: State): bool =
  FORALL (i: sal_Index):
    ((s`sal_values)(i) < 2 AND
      (next`sal_values)(i) = (s`sal_values)(i) + 1) OR
    (NOT ((s`sal_values)(i) < 2) AND
      (next`sal_values)(i) = 0)

IMPORTING SAL_Transition_System[State, initial, step] AS system
sal_aligned: THEOREM system.invariant(
  LAMBDA (s: State): FORALL (i, j: sal_Index):
    (s`sal_values)(i) = (s`sal_values)(j))
```

One quantified relation covers every positive `N`; no theory is generated for each individual cell. Model checking later chooses a finite instance.

N-ary composition: inspect the source-level TCC

```
(defparameter family (sal-doc-nary))  ; generic N
(sal-show-obligations family)
(sal-show-source family "check_step_1_TCC1")
```

Recorded report before proof, with the directory prefix omitted:

```
nary/nary.sal:10 check_step_1_TCC1 [SUBTYPE; NIL]
  SAL: values'[i_2] = values[i_2] + 1
```

Native PVS formula, with the index name shortened and redundant parentheses removed:

```
FORALL (s, next: State, i: sal_Index[sal_N]):
  (s`sal_values)(i) < 2 IMPLIES
    0 <= (s`sal_values)(i) + 1 AND (s`sal_values)(i) + 1 <= 2
```

The renamed array assignment still points to the cell's increment at line 10.

N-ary composition: invoke SAL from the PVS prover

```
;; PVS Lisp: select a finite instance in its own context.
(defparameter three (sal-doc-nary :n 3)) ; nary{3}!sync
(sal-show-obligations three)
```

Prove the source subtype TCC and four imported reachability TCCs with `grind`. Then start `sal_aligned` at its original PVS goal and run:

```
(then (sal-source)
      (sal-check "SAL_sal_nary_sal_sync.aligned"))
```

Recorded: SAL reports `PROVED`; PVS completes the proof (`Q.E.D.`). The five TCCs and saved strategy proof also replay successfully in a fresh PVS process.
This checks `N = 3`. Generic registration is deferred; it does not prove alignment for every `N`. Proof evidence reconstruction remains future work.

N-ary composition: the asynchronous contrast

With `[]`, one chosen component steps and all other components keep their values. The normalized relation uses $\exists i$, the cell step, and $\forall j : \; i = j \vee values'(j) = values(j)$.

```
async_aligned: THEOREM async |-
  G(FORALL(i, j: Index): values[i] = values[j]);
```

```
;; PVS Lisp; typecheck, then prove this instance's typing TCCs.
(defparameter interleaved (sal-doc-nary :n 3 :module "async"))
;; In the sal_async_x5F_aligned proof:
(sal-check "SAL_sal_nary_sal_async.async_aligned")
(sal-show-check)
```

Recorded counterexample: $(0, 0, 0) \to (1, 0, 0)$. The PVS goal stays open. The source trace names component `i = 1` and transition `increment`.
Reproduce both checks and the fresh positive replay: `src/sal/doc/nary/run.sh` (using the tool paths from `doc/README.org`).

Record arrays: native types on both sides

Exact generated declarations, line-wrapped. Types belong to the context theory; `State` belongs to the module theory.

SAL types and state variable

```
Index: TYPE = [0..1];
Small: TYPE = [0..2];
Cell: TYPE =
  [# enabled: BOOLEAN,
     value: Small #];

% Inside machine:
OUTPUT cells: ARRAY Index OF Cell
```

PVS context and module excerpts

```
sal_Index: TYPE = subrange(0, 1)
sal_Small: TYPE = subrange(0, 2)
sal_Cell: TYPE =
  [# sal_enabled: bool,
     sal_value: sal_Small #]

State: TYPE =
  [# sal_cells:
       [sal_Index -> sal_Cell] #]
```

An array is a function over its index type. Record fields remain record fields.

Record arrays: initialization and field update

Excerpts from `record_array`. PVS renames the index to `i` and omits `defined = TRUE`, neutral conjuncts and redundant parentheses.

SAL array expressions

```
INITIALIZATION
  cells = [[i: Index]
    (# enabled := TRUE,
       value := 0 #)]

% THEN branch of the step array:
cells[i] WITH .value :=
  IF cells[i].value < 2
  THEN cells[i].value + 1
  ELSE 0 ENDIF
```

Full model and translation:

counterparts/record-array/

PVS lambda and record update

```
initial(s: State): bool =
  s`sal_cells =
    (LAMBDA (i: sal_Index):
      (# sal_enabled := TRUE,
         sal_value := 0 #))

% Same THEN branch:
(s`sal_cells)(i) WITH
 [sal_value :=
   IF (s`sal_cells)(i)`sal_value < 2
   THEN (s`sal_cells)(i)`sal_value + 1
   ELSE 0 ENDIF]
```

Terminal machine: the missing successor is preserved

Here `defined` is `TRUE`. PVS formulas are shown without neutral conjunctions.

SAL module excerpt

```
OUTPUT value: [0..2]
INITIALIZATION value = 0
TRANSITION
 [ advance: value < 2 -->
     value' = value + 1 ]

bounded: THEOREM machine |-
  G(value <= 2);
```

The reachable prefix is $0 \to 1 \to 2$. At 2, safety holds and progress fails.

Full theory and semantic import:

counterparts/terminal/

PVS module theory excerpt

```
State: TYPE =
  [# sal_value: subrange(0, 2) #]
initial(s: State): bool =
  s`sal_value = 0
step(s, next: State): bool =
  s`sal_value < 2 AND
  next`sal_value = s`sal_value + 1

sal_bounded: THEOREM
  system.invariant(LAMBDA (s: State):
    s`sal_value <= 2)
```

The exported `idle` relation is not added to `step`.

Progress and causality ask different questions

Native mapped-axiom TCCs ask:

- Does an initial state exist?
- Does every reachable state have a successor?

These companion obligations are generated by default. Use `:progress-checks nil` to omit them.

`:causality-checks t` adds a rank obligation for supported deterministic, whole-variable definitions.

`request -> enabled -> acknowledged` has a schedule.

`x = y; y = x` has a cycle, even though Boolean fixed points exist.

Ask SAL about a deadlock; ask PVS for the contract

```
;; Diagnostic API, independent of a proof:
(defparameter terminal
  (sal-doc-open "integration/terminal_deadlock.sal"
                "machine" "terminal"))
(sal-run-check (sal-transition-view-source terminal) nil
  :backend :deadlock :module "machine")
(sal-print-check-result)
```

- `sal-check` and `sal-check-pvs` close admitted safety invariant goals.
- Deadlock detection is diagnostic; native progress TCCs are separate goals.
- Structural acyclicity is not a general fixed-point existence or uniqueness decision procedure.

Try integration/terminal_deadlock.sal and causality_good.sal / causality_cycle.sal.

Prove the original SAL safety theorem from PVS

```
;; PVS Lisp: reopen the positive SAL example.
(defparameter v
  (sal-doc-open "bounded_counter.sal" "counter" "counter-proof"))
```

Discharge its native typing TCCs and start the generated `sal_bounded` theorem. At its original prover goal:

```
(sal-source)
(sal-check "SAL_sal_bounded_x5F_counter_sal_counter.bounded")
(sal-show-check)
```

`sal-source` displays the original SAL theorem and location. The registered goal closes on an admitted checker proof; transformed sequents remain PVS.

Turn around: start with an ordinary PVS library

```
Reverse_Counter_Library[limit: posnat]: THEORY
BEGIN
  Count: TYPE = {n: int | 0 <= n AND n <= limit}
  State: TYPE = [# count: Count, started: bool #]

  advance(n: Count): Count =
    IF n < limit THEN n + 1 ELSE n ENDIF
  next(s: State): State =
    (# count := advance(s`count),
       started := IF s`count < limit
                  THEN TRUE ELSE s`started ENDIF #)
  % Predicates follow on the next slide.
END Reverse_Counter_Library
```

The source library has no SAL import. Its definitions own the behavior.

State the machine and its safety property in PVS

```
initial(s: State): bool = s`count = 0 AND NOT s`started
step(s, t: State): bool = t = next(s)
safe(s: State): bool = s`count = 0 OR s`started
zero(s: State): bool = s`count = 0
```

For `limit = 3`, the reachable behavior is:

$$(0, \text{FALSE}) \longrightarrow (1, \text{TRUE}) \longrightarrow (2, \text{TRUE}) \longrightarrow (3, \text{TRUE})$$

The last state can repeat. `safe` is invariant; `zero` fails after one step.

examples/reverse/Reverse_Counter_Library.pvs; predicates excerpted

Interpret the transition view over those definitions

```
Reverse_Counter: THEORY
BEGIN
  IMPORTING Reverse_Counter_Library[3] AS lib
  native: THEORY = SAL_Transition_View{{
    S := lib.State, initial := lib.initial, step := lib.step }}

  safe: THEOREM native.invariant(lib.safe)
  stays_zero: THEOREM native.invariant(lib.zero)
```

`SAL_Transition_View` maps `S`, `initial` and `step` into the same `SAL_Transition_System` semantics used by the SAL frontend.
There is one concrete machine definition. The interpretation selects its names and parameters; it does not restate its transition relation.

Extract the actual typed theorem

```
;; PVS Lisp; copies examples and typechecks in doc/.demo/native/:
(sal-doc-native "Reverse_Counter")
(defparameter extracted
  (sal-extract-pvs "Reverse_Counter" "safe"))
(sal-show-pvs-extraction extracted)
```

The extractor follows resolved declarations and native theory substitution to recover the carrier, initial predicate, step relation and property.
It admits a concrete finite dependency slice and checks the emitted SAL with the native SAL frontend. It never accepts a caller's arbitrary theorem pairing.

PVS to SAL: the concrete carrier and initial state

The library is instantiated at `limit = 3`. The SAL output below only changes line wrapping and redundant parentheses; full files are in `counterparts/reverse/`.

Original PVS library excerpts

```
Count: TYPE =
  {n: int | 0 <= n AND n <= limit}
State: TYPE =
  [# count: Count, started: bool #]

initial(s: State): bool =
  s`count = 0 AND NOT s`started
```

The finite carrier keeps the full subtype predicate. Names are encoded in SAL.

Extracted SAL excerpts

```
State: TYPE =
 [# pvs_count:
      {pvs_subtype_1: [0..3] |
        0 <= pvs_subtype_1 AND
        pvs_subtype_1 <= 3},
    pvs_started: BOOLEAN #];

% Inside machine:
LOCAL state: State
INITIALIZATION
 state IN {candidate: State |
   candidate.pvs_count = 0 AND
   NOT candidate.pvs_started}
```

PVS to SAL: the concrete step and invariant

The same extraction inlines `next` and `advance`. Parentheses are reduced for display.

Original PVS library excerpts

```
advance(n: Count): Count =
 IF n < limit THEN n + 1
 ELSE n ENDIF
next(s: State): State =
 (# count := advance(s`count),
    started :=
      IF s`count < limit THEN TRUE
      ELSE s`started ENDIF #)
step(s, t: State): bool =
  t = next(s)
safe(s: State): bool =
  s`count = 0 OR s`started
```

At count 3, the PVS next function repeats the state. SAL preserves that step.

Extracted SAL excerpts

```
TRANSITION
 state' IN {candidate: State |
   candidate =
     (# pvs_count :=
          IF state.pvs_count < 3
          THEN state.pvs_count + 1
          ELSE state.pvs_count ENDIF,
        pvs_started :=
          IF state.pvs_count < 3
          THEN TRUE
          ELSE state.pvs_started ENDIF #)}

safety: THEOREM machine |-
 G(state.pvs_count = 0 OR state.pvs_started);
```

Close the native PVS theorem with a SAL strategy

After proving the native TCCs in the theorem and its imports, start `safe` in the PVS prover. At its original singleton goal:

```
(sal-check-pvs "Reverse_Counter.safe")
;; Or finite k-induction:
(sal-check-pvs "Reverse_Counter.safe" :backend :bmc :depth 4)
(sal-show-check)
```

Recorded result: SAL reports `proved.` and PVS reaches `Q.E.D.` for both routes. SMC is exhaustive. Plain bounded absence of counterexamples is not a proof. The proof rule invokes BMC with induction and rejects inexact results.

A false property produces a usable failure

```
;; PVS Lisp, before starting the theorem:
(sal-extract-pvs "Reverse_Counter" "stays_zero")
```

In the `stays_zero` proof:

```
(sal-check-pvs "Reverse_Counter.stays_zero")
(sal-show-check)
```

Recorded result: `COUNTEREXAMPLE`; the PVS goal remains open.
The violating prefix reaches `count = 1` from `count = 0`. The complete native SAL output is retained for inspection; source field names are encoded in SAL.

Nondeterminism and terminal states survive extraction

```
BEGIN
  State: TYPE = [# done: bool, seen: bool #]
  initial(s: State): bool = NOT s`done AND NOT s`seen
  step(s, t: State): bool = NOT s`done AND t`seen
  safe(s: State): bool = s`done IMPLIES s`seen
```

The step constrains `t‘seen` but leaves `t‘done` nondeterministic. Once done is true, there is no successor.

Check	Recorded result	Meaning
observed_before_finish	Proved	Safety includes terminal states
never_finishes	Counterexample	Finishing remains possible
Deadlock diagnostic	Deadlock	No implicit stuttering was added

Finite functions become SAL arrays

```
Index: TYPE = {i: int | 0 <= i AND i <= 1}
State: TYPE = [# cursor: Index, marked: [Index -> bool] #]

initial(s: State): bool =
  s`cursor = 0 AND (FORALL (i: Index): NOT s`marked(i))

% In next(s), the updated field is:
% marked := s`marked WITH [(s`cursor) := TRUE]

ordered(s: State): bool = s`marked(1) IMPLIES s`marked(0)
```

`Reverse_Array.mark_zero_first` proves by `sal-check-pvs`. The example exercises finite quantification, selection and direct array update.

Scalar enums and tuples need no deep encoding

```
Phase: TYPE = {idle, active, done}
State: TYPE = [Phase, bool]
initial(s: State): bool = s`1 = idle AND NOT s`2
step(s, t: State): bool =
  (s`1 = idle AND t = (active, TRUE)) OR
  (s`1 = active AND t = (done, s`2))
safe(s: State): bool = s`1 = done IMPLIES s`2
```

`Reverse_Phase.completed_after_active` proves through SAL.

PVS tuple projections become SAL tuple projections, and the finite enum becomes a SAL scalar type. The `done` state remains terminal.

Direct parameterization and theorem aliases work too

```
IMPORTING Reverse_Counter_Library[1] AS lib
IMPORTING SAL_Transition_System[
  lib.State, lib.initial, lib.step] AS native
safe: THEOREM native.invariant(lib.safe)
```

No facade is required for `Reverse_Direct.safe`. The bound `1` is a different concrete instance from the earlier bound `3`.

```
safe_statement: bool = native.invariant(lib.safe)
safe_alias: THEOREM safe_statement
```

Definitional wrappers unfold through PVS resolution. A similarly named, unrelated `invariant` function is rejected.

Native typing obligations cannot be bypassed

```
State: TYPE = {n: int | 0 <= n AND n <= 3}
next(s: State): State = s + 1
```

At `s = 3`, the result is outside `State`. PVS retains the false range TCC.

```
(sal-doc-native "Reverse_Admission")
(sal-extract-pvs "Reverse_Unproved_TCC" "candidate")
;; Then, in the candidate proof:
(sal-check-pvs "Reverse_Unproved_TCC.candidate")
```

Recorded behavior: the rule refuses before invoking SAL because native TCCs remain unproved. Typechecking alone is insufficient.

Interpretations retain mapped-axiom TCCs

```
% An abstract contract:
preserves_state: AXIOM
  FORALL (s, t: S): step(s, t) IMPLIES s = t

% Interpret it over the existing counter:
contract: THEORY = Reverse_Stuttering_Contract{{
  S := lib.State, initial := lib.initial, step := lib.step }}
```

The counter changes state, so this interpretation has an unproved mapped-axiom TCC. Its separate `safe` predicate is still true.

`Reverse_Mapped_TCC.candidate` is refused before checker invocation. The integration does not turn an abstract axiom into a free assumption.

Admission is explicit and fails closed

Admitted reverse fragment	Explicit refusal examples
Boolean and bounded integer carriers	Unbounded integer state
Records, tuples, scalar enums	Dependent data, opaque types
Finite arrays and quantifiers	Uninterpreted functions
Pure specialized definitions	Recursion and fixed points
Audited Boolean/integer operations	DIV/MOD, real arithmetic
Concrete invariant theorem	Generic or unrelated theorem

Integer bounds come from subtype conjunctions; all predicates are retained. No guessed bound or bounded approximation makes a fragment admissible.

Executable rejection cases: examples/reverse/Reverse_Admission.pvs

The proof rule binds one theorem to one extraction

Before invoking SAL, the rule requires:

1. The registered, typechecked declaration and its original singleton goal.
2. Unchanged PVS/SAL sources and SAL configuration.
3. Complete native TCC proofs, including interpretation obligations.
4. An admitted backend and an exact semantic result.

An expanded/altered goal, a stale import, a modified generated SAL file, unknown output or an approximation warning leaves the goal open.
First extraction inherits native PVS freshness checks, supplemented by a stored digest when available. After registration, exact content snapshots apply.

Save the proof, then replay from a fresh PVS process

```
src/sal/tools/run-reverse-tests.sh
src/sal/tools/run-parameter-tests.sh
src/sal/tools/run-bridge-tests.sh
```

The runners configure extension-local contexts and retain logs and `.prf` files. The reverse replay re-extracts each target before replaying its saved strategy.

Validated suite	Main assertions	Replay assertions
Reverse path	92	84
Parameters and dependencies	102	10
Existing bridge	90	38

These are regression assertion counts, not independent theorem counts.

What the broader SAL corpus result establishes

The recorded fresh frontend run typechecked 372/372 targets:

- All 210 native-accepted source files in `~/git/kn-sal/examples`.
- Two documented template instances.
- Module declarations and library-only contexts, with symbolic parameters.

Six raw inputs remain explicit native rejections: three empty XML artifacts, one legacy dialect source, and two uninstantiated templates.
This is frontend coverage. It does not prove every emitted TCC or source property, or model-check every possible concrete instance.

Coverage unit, inventory and retained failures: tests/corpus.md

Explore the remaining frontend demonstrations

Feature	Example / test entry
Record arrays, nested updates, lambdas	integration/record_array.sal
Subtypes and nondeterministic selection	integration/predicate_subtype.sal
Recursive definitions and termination TCCs	integration/frontend_nonterminating.sal
CTL and terminal-state semantics	integration/frontend_semantics.sal
RINGSET bounds and wraparound	parameters/parameter_ring_parent.sal
Temporal macros and call-domain TCCs	parameters/parameter_temporal_parent.sal
Shared LET helpers for large expressions	integration/frontend_let_blocks.sal
Explicit representation correspondence	integration/PVS_Library_Adapter.pvs

Frontend translation support is broader than trusted checker admission. For example, unaudited DIV/MOD semantics defer automatic registration.

Does this recover full predicate subtyping?

The shallow representation supports PVS-strength predicates in the translated pure fragment. Membership becomes a proof obligation.

- Arbitrary refinements need not be decidable or finite.
- All relevant native TCCs must be discharged for completed typing assurance.
- Whole-SAL coverage also requires preserving every source typing occurrence, exact carriers, primitives, parameters and imports.

The result to aim at

Exact source verification conditions, with type soundness when their PVS proofs are complete. Automatic proof completeness is impossible in general.

Detailed definitions and proof: doc/semantics/README.org

What the PVS semantics report establishes

Owre and Shankar, *The Formal Semantics of PVS*, CSL-97-2R, revised March 1999. The supplied `semantics.pdf` is the reference.

$$\tau_\Gamma(e) = T \ \wedge \ \gamma \models \Gamma \quad \Longrightarrow \quad M_{\Gamma,\gamma}(e) \in M_{\Gamma,\gamma}(T).$$

Theorem 9 combines type, term and proof soundness. The report also states that type correctness is undecidable; it does not establish proof completeness. Arithmetic, recursive definitions and modern interpretations require additional justification beyond that report's core.

Report: Theorem 9, p. 60; undecidability, p. 29; exclusions, pp. 2–3, 48. Printed pages; PDF pages are five larger.

Define SAL types as sets

$$[\![\{x : T \mid p\}]\!]_\eta = \{v \in [\![T]\!]_\eta \mid [\![p]\!]_{\eta[x \mapsto v]} = \text{true}\},$$

$$[\![A \times B]\!]_\eta = [\![A]\!]_\eta \times [\![B]\!]_\eta,$$

$$[\![[A \to B]]\!]_\eta = \{f : [\![A]\!]_\eta \to [\![B]\!]_\eta\}.$$

Records are labelled products; arrays are functions with exact domains. Application outside the domain is undefined in the source semantics.
Type predicates must themselves be well formed. An empty subtype is allowed; introducing a required witness adds a nonemptiness obligation.

Source analogue of Definition 17, p. 27. Scope: resolved, nonrecursive pure core with explicitly matched primitive meanings.

Preserve the domain when removing refinements

Let $\mu(T)$ be the maximal carrier and π_T its membership predicate.

$$\mu(\{x : T \mid p\}) = \mu(T), \qquad \pi_{\{x:T|p\}}(v) = \pi_T(v) \wedge p[v/x],$$
$$\mu([A \to B]) = [A \to \mu(B)], \quad \pi_{[A\to B]}(f) = \forall a : A.\ \pi_B(f(a)).$$
$$[\![T]\!] = \{v \in [\![\mu(T)]\!] \mid \pi_T(v)\}.$$

For an array indexed by `[1..N]`, the maximal type still has that domain. Erasing it to all integers would change the model.

Definitions 10 and 12, pp. 21, 23; Proposition 3.9, pp. 27–28. Carrier lemma: structural induction on source types.

A sound and complete subtype condition

For well-formed S, T with a common maximal carrier:

$$\boxed{[\![S]\!] \subseteq [\![T]\!] \quad \Longleftrightarrow \quad \models \ \forall v : \mu(S).\ \pi_S(v) \Rightarrow \pi_T(v)}$$

Proof. Rewrite membership on both sides with the carrier lemma. The left side then says precisely that every value satisfying π_S also satisfies π_T.

This is semantic relative completeness: a validity oracle would settle every compatible subtype check. It is not completeness of PVS proof search.

Different maximal carriers first need the report's compatibility and type-equivalence obligations, or an explicit representation correspondence.

Commands: relation and source typing are separate

$$R_{g,x,e}(s,t) \equiv g(s) \wedge t = s[x \mapsto e(s)].$$

$$\mathrm{VC}_{x:=e} \equiv \forall s : \mathrm{TypedStore}. \quad g(s) \Rightarrow e(s) \in [\![T_x]\!].$$

Also check the guard itself, RHS definedness and assignment addresses. Each deterministic assignment is checked independently.

For `x' IN {y:A | p}`, values come from the intersection of the target type with the offered set. This does not require all offered values to fit, or promise any successor.

The obligations quantify over all typed states, including unreachable ones.

Contextual expressions; independent command choices

Source form	Context for the later expression
`IF g THEN a ELSE b`	g for a; $\neg g$ for b
`a AND b`	a for b
`a OR b`	$\neg a$ for b
`a IMPLIES b`	a for b
`c1 [] c2`	Check both command arms independently

Keep each Boolean expression's contextual typing. A competing transition must not justify a guard's function-domain condition.
Logical equivalence of relations alone is too weak to justify preservation of source typing obligations.

PVS contextual typing: Chapter 6, pp. 49–52. The command-choice boundary is specific to the SAL source typing contract.

Exactness of the source-core VC system

Define `LocalSafe` using independent source evaluation: all typing sites denote and satisfy their required types under their lexical paths.

$$\boxed{\text{LocalSafe}_S(C) \quad \Longleftrightarrow \quad \models_S \text{VC}_S(C)}$$

Scope: structurally compatible types, resolved acyclic declarations, pure nonrecursive expressions, exact function domains and matched primitives.
Proof: mutual structural induction. Application uses domain membership; IF splits on its test; assignment uses the carrier lemma; choice checks each arm; N-ary composition universally quantifies the component conditions.

Full rules and both directions of the argument: doc/semantics/README.org. This mathematical core result is not a verification of the Lisp compiler.

Relate the SAL core to the PVS set semantics

Let E be the shallow translation; ρ renames fields and lifts corresponding values to products and exact-domain functions.

$$\rho_T\big[\![T]\!]_S\big] = M_P(E(T)), \qquad \rho_T([\![e]\!]_S) = M_P(E(e)).$$

$$\mathrm{LocalSafe}_S(C) \iff \models_P E(\mathrm{VC}_S(C)).$$

Structural induction proves preservation for well-formed types and locally safe expressions, assuming capture avoidance and primitive correspondence.
Sound PVS proofs of these obligations imply source-local type safety. Validity does not imply provability or automatic discharge.

The checked semantic core uses unrestricted stores

```
Store: TYPE = [I -> V]
typed(s: Store): bool = FORALL (j: I): ok(j, s(j))

atom(i: I, g: [Store -> bool], e: [Store -> V])
    (s, t: Store): bool =
  g(s) AND t = s WITH [(i) := e(s)]
```

ok is an arbitrary predicate. The successor store is not restricted to already-typed values, so the preservation theorem can detect a bad update.
The abstract functions g and e are total. Source guard/RHS definedness must be established before applying this theorem to an elaborated command.

doc/semantics/Typing_Core.pvs; semantic metatheory, not a runtime embedding.

A mechanized assignment theorem and composition laws

$$\mathrm{preserves}(R) \equiv \forall s, t.\ \mathrm{typed}(s) \wedge R(s,t) \Rightarrow \mathrm{typed}(t).$$

$$\mathrm{preserves}(\mathrm{atom}(i,g,e)) \iff \forall s.\ \mathrm{typed}(s) \wedge g(s) \Rightarrow \mathrm{ok}(i, e(s)).$$

Choice preservation reduces to preservation of every arm. Indexed local obligations quantify over all component indexes. Simultaneous pointwise updates are sound when each owned coordinate satisfies its predicate.

The synchronous converse is not claimed: a blocked peer can hide an unsafe local update. These lemmas use ordinary PVS proofs, without the SAL rule.

Replay the six semantic-core proofs

```
# Same PVS_BIN, SAL_HOME and SAL_XMLS_SOURCE as the tutorial:
src/sal/doc/semantics/run-typing-core.sh
```

Group	Checked results
Local update (2)	Update membership; assignment equivalence
Binary choice (1)	Both arms preserve typing
Indexed choice (2)	Component preservation; exact local VCs
Simultaneous update (1)	Coordinate checks suffice

Verified: six proved-complete declarations, zero TCCs, with fresh replay and complete proof chains. No added axioms or SAL proof rule.

Saved proofs and verification record: doc/semantics/Typing_Core.prf, typing-core-evidence.txt and typing-core-proofchain.txt.

Apply the result to the N-ary counter

For parameter $N > 0$, use $I = [1..N]$, $V = \mathbb{Z}$ and $\mathrm{ok}(i, v) \equiv 0 \le v \le 2$.

$$\forall i : I,\ s : \mathrm{TypedStore}. \quad s(i) < 2 \Rightarrow 0 \le s(i) + 1 \le 2.$$

The wrap branch writes zero. Each coordinate preserves its refinement; the independently owned simultaneous update preserves the array type.
The earlier `sal-check` example then checks the concrete instance's alignment invariant. That property is additional to the typing theorem; arbitrary predicates and symbolic N do not imply finite checker admission.

Connect this rule to doc/nary/nary.sal and its source-level increment TCC. The correspondence is worked mathematically, not a compiler-certification theorem.

Audit example: a guard obligation must survive

```
Positive: TYPE = {n: INTEGER | n > 0};
positive(p: Positive): BOOLEAN = p > 0;
% arg: INTEGER; flag: BOOLEAN
TRANSITION
 [ free: TRUE --> flag' IN {b: BOOLEAN | TRUE}
 [] guarded: positive(arg) --> flag' = FALSE ]
```

Before the repair, the first arm made the operational OR true and masked the later guard's subtype TCC. The diagnostic reported zero source obligations.
Independent arm checks now retain this false obligation. It stays open; the relation is unchanged.

```
check_choice_step_2_TCC1: OBLIGATION
  FORALL (s: State): s`sal_arg > 0
```

Reproducer: doc/semantics/audit_guard.sal. The index positive control already retained its expected domain TCC.

What remains to prove about the implementation

1. Resolution/export/normalization preserves source occurrences and contexts.
2. The renderer preserves carriers, primitive meanings, binding and imports.
3. Native TCCs, including trivial/subsumed ones, realize exactly the core VCs.
4. Semantic extensions and theory interpretations meet their own obligations.

Under those premises, completed sound PVS proofs imply source-local typing. The subtype equivalence also gives semantic relative completeness.
The corpus tests and the checked update laws support this work; they do not establish all four premises for every SAL feature.

Type safety does not settle progress or reachability

- An unreachable `x = 2 --> x' = 3` still fails local typing for `x:[0..2]`.
- An impossible typed initial relation can hide an invalid initializer.
- A synchronous deadlock can mask a component's invalid update.
- Well-typed algebraic equations need not have a solution, or a unique one.

Keep initialization feasibility, deadlock freedom and algebraic-loop contracts separate from predicate-subtype correctness.

What "complete" means here

The generated conditions exactly characterize the specified local typing contract, relative to semantic validity and structural compatibility.

The boundary to keep visible

Available now

Readable shallow theories, native TCCs with source origins, concrete PVS definition extraction, trusted SAL invariant strategies, retained counterexamples, and saved-proof replay.

Deliberately still future work

Reconstruction of SAL proof evidence inside PVS; broader reverse and temporal fragments; full source-language rendering of transformed sequents; general fixed-point causality analysis.

There is no logical interpreter layer. Literal zero cost is not promised: elaboration, specialization, inlining and backend checking still do work.

Three experiments to try next

1. In the copied `.demo/native/Reverse_Counter.pvs`, change the bound `3`. Retypecheck, inspect TCCs, re-extract, and prove `safe` again.
2. Replace `safe` by `zero`. Follow the retained counterexample back to the native PVS transition.
3. Introduce the overflowing `next(s) = s + 1` definition. Observe the native TCC and the checker's refusal before it runs.

The working loop is:

$$\text{define} \longrightarrow \text{typecheck / inspect TCCs} \longrightarrow \text{extract} \longrightarrow \text{prove or diagnose}$$

Start with doc/README.org; implementation APIs and limits are in ../README.md.